- gives 0 if str1 = str2
- gives a positive result if str1 > str2, i.e. if str1 comes after str2 in alphabetic order

We will now create a little program that prompts the user for two names and prints them in alphabetic order:

```
#include <iostream.h>
#include <string.h>
void main()
{
  char cName1[30], cName2[30];
  cout << "Enter a name: ";
  cin >> cName1;
  cout << "Enter another name: ";
  cin >> cName2;
  if (strcmp(cName1,cName2) < 0)
    cout << cName1 << endl << cName2;
  else
    cout << cName2 << endl << cName1;
  cout << endl;
}
```

First we declare two string arrays, cName1 and cName2, which can contain max 29 characters each (pos 30 is for the null character). The user enters two names. A name must not contain blanks. How would you do to allow blanks?

The if statement checks if cName1 comes before cName2 in alphabetic order. If so, cName1 is printed first and then cName2. Otherwise the names are printed in reversed order.

5.13 Copying Strings

In the Arrays chapter we copied an array's values to another array by copying each item singly. For strings there is a special function, strcpy(), that copies the entire string to another string. If for instance the string array str2 contains the string 'John Smith' and we execute the statement:

```
strcpy(str1, str2);
```

then str1 will also contain the string 'John Smith'.

5.14 Array with String Arrays

Suppose that we want to store a number of names in an array, where each name is itself a string array. Then we will need a two-dimensional array. Below we declare a two-dimensional array with space for 5 names with 30 positions each (29 characters plus the null character):

```
char cNames[5][30];
```


Download free eBooks at bookboon.com

This two-dimensional string array could be represented like this:

	0	1	2	3	4	5	…
0	J	o	h	n	\0		
1	E	d	w	a	r	d	\0
2	B	o	b	\0			
3	E	v	E	\0			
4	A	d	a	m	\0		

To print one of the names, e.g. the third name (indes=2), we code:

```
cout << cNames[2];
```

Note that when we print or read a sequence of characters to a string array, we don't need to specify the index. The program prints the characters from the beginning of the string until the null character is found. However, for a two-dimensional array, we must specify which of the names to be printed, i.e. we must specify the first index. In the example above we used indes=2, which gives the printout 'Bob'.

To print a single character from the matrix we must use both indeces. The statement:

```
cout << cNames[1][4];
```

prints the character with index 4 from the name with index 1, i.e. 'r' in 'Edward'.

5.15 Sorting Strings

We want to sort the string matrix above, i.e. the names should be reorganized into alphabetic order. Do you remember the sorting algorithm from the Arrays chapter? If no, check it out. There we compared the items in pairs and interchanged their positions if the right item was less than the left. The logic is the same for string arrays, but we will have to use our special string functions to be able to compare and copy strings.

We first prompt the user for five names to be stored in the string matrix. Then we sort the strings in a double loop, and finally we print the sorted list of names:

```
#include <iostream.h>
#include <string.h>
void main()
{
  char cNames[5][30], temp[30];
  cout << "Enter 5 names. Press Enter after each" << endl;
  for (int i=0; i<5; i++)
    cin >> cNames[i];
  for (i=0;i<4;i++)
    for (int j=i+1; j<5; j++)
```


Download free eBooks at bookboon.com

```
        if (strcmp(cNames[i],cNames[j]) > 0)
        {
          strcpy(temp,cNames[i]);
          strcpy(cNames[i],cNames[j]);
          strcpy(cNames[j],temp);
        }
   cout << endl << "Sorted names:" << endl;
   for (i=0; i<5; i++)
     cout << cNames[i] << endl;
}
```

First we declare a two-dimensional string array named cNames, and a string array named temp, which we will use for the triangular exchange inside the double loop.

The first for-loop reads five names to the string matrix cNames.

Then we have the double loop. The outer for-loop goes from 0 to 3 (next last name) with the loop counter i. The inner for-loop has the loop counter j which goes from 1 greater than i to 4 (last name).

The if statement compares the two names indicated by the indeces i and j. If the “left” name (i) is greater than the “right” (j), then cNames[i] comes after cNames[j] in alphabetic order, and they should interchange their positions.

The interchange is made by means of strcpy() by copying cNames[i] to the temporary string array temp. Then we copy the name in position j to position i. Finally we copy the temp name (the old name in position i) to position j. The names now have been interchanged in the string matrix.

Download free eBooks at bookboon.com

At completion of the double loop, the names have been sorted in alphabetic order and we can with the last for-loop print one name at a time from position 0 to 4.

5.16 Substring

Sometimes you will need to extract a number of characters from a string. Then we will get use of the strncpy() function, which copies a given number of characters from one string to another. Example:

```
strncpy(cStr1, cStr2, iNo);
```

This statement copies the number of characters given by the variable iNo counted from the beginning of the string cStr2 and stores this substring in cStr1.

If you want to copy a number of characters from a given position and not from the beginning of cStr2, use this variant:

```
strncpy(cStr1, cStr2 + i, iNo);
```

This statement copies the number of characters given by the variable iNo counted from position i of cStr2 and stores this substring in cStr1.

Here we for the first time get in touch with the similarity between strings and **pointers**. A pointer is a variable that points to a certain memory address. When we write cStr2, it is interpreted as a pointer that points to the string starting with the first character of cStr2, i.e. with index=0. If we write cStr2+1, it is interpreted as a pointer that points to the character with index=1 in cStr2. If we write cStr2+i, it is interpreted as a pointer that points to the character with index=i of cStr2. Enough with pointers for this time.

We will get use of this function in subsequent program examples.

5.17 Concatenating Strings

It is possible to add strings together (concatenate) with the strcat() function. The statement

```
strcat(cStr1, cStr2);
```

concatenates cStr1 and cStr2 and puts the result in cStr1, i.e. cStr2 is added at the end of cStr1.

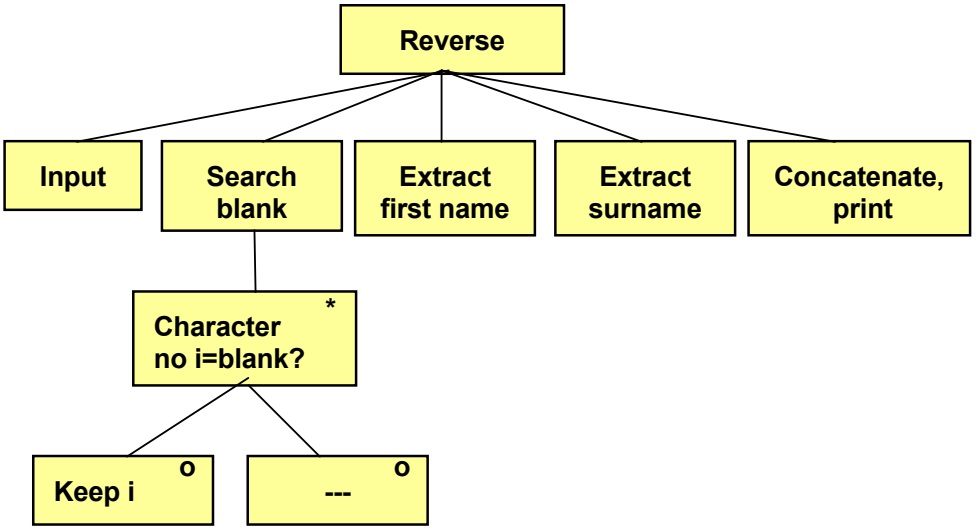
We will get use of this function in subsequent program examples.

5.18 Interchanging First Name and Surname

We will now create a program that prompts the user for his first name and surname separated by a blank. The program will then interchange them so that the surname is printed first.

The way of doing this is to search for the position of the blank. With the strncpy() function we can then separate the first name and surname, and then with the strcat() function put the substrings together with the surname first. We start with a JSP graph:

Download free eBooks at bookboon.com



First we read the name from the user input. Then we search for the position of the blank by looping through the characters and checking if any of them is blank. When a blank is found we save its position.

By means of the position of the blank we can now extract the two substrings consisting of first name and surname. Finally we put them together in reversed order and print them.

Here is the code:

```
#include <iostream.h>
#include <string.h>
void main()
{
  char cName[30], cFirst[15], cSur[30];
  cout << "Enter your name: " << endl;
  cin.getline(cName,29);
  for (int i=0; i<29; i++)
    if (cName[i] == ' ')
      break;
  strncpy(cFirst,cName,i);
  cFirst[i] = '\0';
  strcpy(cSur, cName + i + 1);
  strcat(cSur, " ");
  strcat(cSur, cFirst);
  cout << cSur << endl;
  cout << strrev(cSur) << endl; //reversed name
}
```


Download free eBooks at bookboon.com

First we declare a string array called cName which will contain the name entered by the user (both first and surname). The string array cFirst will be used for the first name and the string array cSur to the surname. The reason for using 30 positions instead of 15 for the surname is that we will concatenate the final result in that string array.

The user is prompted for his name to be stored in the array cName.

The for-loop searchees for the blank. It goes from 0 to 28, i.e. it takes character by character from the name. If a character equals blank, we interrupt the loop with break, which has the effect that the loop counter i is not incremented any more. If the user enters 'John Smith', i will have the value 4 when the loop is interrupted, and that is the position of the blank.

The function strncpy() then copies the first i characters from cName to cFirst. In the case of John Smith the first 4 characters will be copied, i.e. 'John'.

The string cFirst is completed with a null character as the fifth character (index=4).

Then we use the function strcpy() to copy the characters from cName to cSur starting on position i+1. cName+i+1 is interpreted as a pointer that points to i+2nd character of cName. In the case of John Smith i is =4. cName+i+1 consequently points to the string starting in position with index 5, i.e. 'Smith'. Note that the copy is performed character by character until the null character is found.

Then we use the strcat() function to concatenate the surname with the string ' ', i.e. we add a blank to the surname. Then we concatenate surname + blank with the first name. The string cSur now contains surname + blank + first name, which is printed.

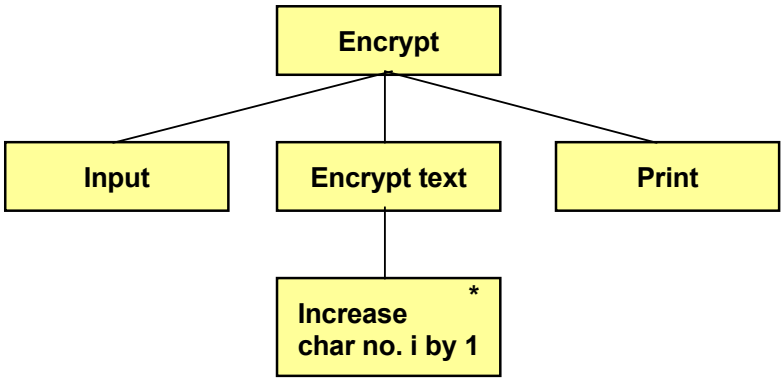
The last statement has actually nothing to do with our task. We have only included it for curiosity. It prints the entire name in reversed order. The function strrev() reverses a string.

Download free eBooks at bookboon.com

5.19 Encryption

We will now write a program that encrypts a text, i.e. distorts it to unreadability. The way of doing this is to take character by character of the text entered by the user and increase the character code by 1. A becomes B, B becomes C etc. This is a very simple encryption algorithm, but gives a hint about how different encryption algorithms work when encrypting mail and other information.

We start with a JSP graph:



The user is first prompted for a text. The encryption takes character by character from the string array and increases the character code by 1. Finally we print the encrypted message. Here is the code:

```
#include <iostream.h>
#include <string.h>
void main()
{
  int iLen;
  char cName[30], cEncrypt[30];
  cout << "Enter your name: " << endl;
  cin.getline(cName,29);
  iLen = strlen(cName);
  for (int i=0; i<iLen; i++)
    cEncrypt[i] = cName[i] + 1;
  cEncrypt[iLen] = '\0';
  cout << "Encrypted: " << cEncrypt << endl;
}
```

We declare the string array cName to be used for the entered name, and cEncrypt to be used for the encrypted text. The user is prompted for his name, which is stored in the string array cName. The length of the text is calculated and stored in the variable iLen.

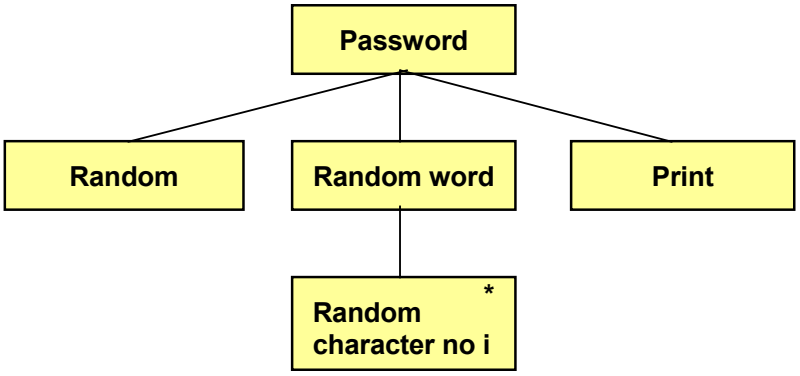
The for-loop goes from the first to the last character of the entered string and increases the character code for each character with 1. The character is stored in the string array cEncrypt.

Download free eBooks at bookboon.com

As the last character we insert the null character at the correct position in cEncrypt, which then is printed.

5.20 Random Password

Passwords should, as you probably know, be difficult to guess to prevent unauthorized access to a computer system. A good way of creating passwords is to let the computer randomly create them. We will write a program that makes the computer able to create passwords only consisting of upper case letters (A-Z). This is of course a limitation, but indicates the logic to be followed. You can then improve it to include lower case letters and digits. We start with a JSP graph:



We begin with creating a random length of the password, which we in this example will limit to between 5-7 characters. Then we create as many characters in the interval A-Z (character codes 65-90). The password is then printed. Here is the code:

```
#include <iostream.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
void main()
{
  int iLen, i;
  char cPw[30];
  srand(time(0));
  iLen = rand() % 3 + 5;
  for (i=0; i<iLen; i++)
    cPw[i] = rand() % 26 + 65;
  cPw[iLen] = '\0';
  cout << "Password: " << cPw << endl;
}
```

We declare the loop variable i, the variable iLen to contain the length of the password, and the string array cPw to contain the random password. We have declared 30 characters, but 9 would suffice in this example.

Download free eBooks at bookboon.com

The random number generator is initiated with the function srand(). Then we create the random length of the password with the modulus operator %, which gives a length in the interval 5-7. rand()%3 gives a random number in the interval 0-2. By adding 5 we transform the interval to 5-7. The number is stored in the variable iLen.

The for-loop then goes from 0 to iLen-1, i.e. as many turns as the number of letters in the password. For each turn of the loop a random number in the interval 65 to 65+25 is created, which corresponds to the character codes for A-Z. Each character code is stored in the string array cPw at position i, i.e. cPw is built up with one more character for each turn of the loop. At the end of cPw we add the null character. cPw is then printed.

5.21 Translation Table

We will now show another method of encrypting a message, namely by means of a translation table. It consists of a set of the characters A-Z, very well mixed up. Thus, the characters are not in alphabetic order. We use the random number generator to mix the characters.

Suppose we have got the following translation table:

```
DGZCW...
```

where D is in position 0, G in position 1 etc. We use the table to translate a A to D, B to G etc. each character in the original message will consequently be translated to another character:

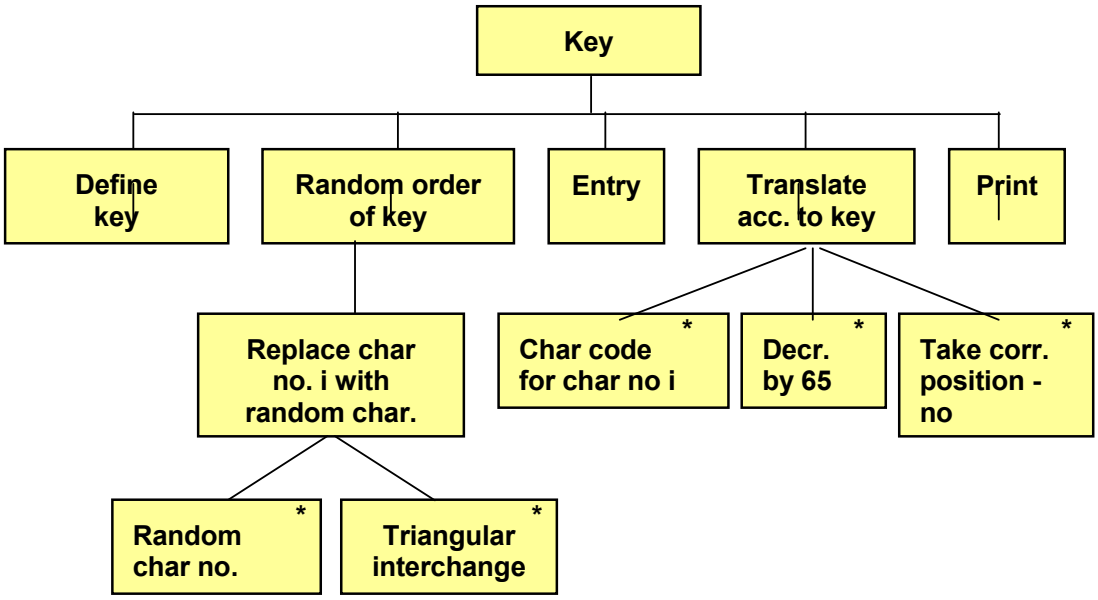
ABCDE...

Click on the ad to read more

Download free eBooks at bookboon.com

DGZCW...

The JSP graph below shows the logic:



'Define key' means that we define a string containing the characters A-Z.

'Random order of key' implies that we mix the characters of the key, which is done by looping through the characters and create a random position for another character of the key. These two characters will then interchange their positions. In that way all characters will interchange its position with some other character.

The user is then prompted for the message in upper case.

'Translate acc. to key' is made by taking the character code for one character at a time in the entered message and decreasing the code by 65. A will then get the value 0, B the value 1 etc. This value gives the index in the key for the character to substitute the original character.

If we use the key:

```
DGZCW...
```

like in the introductory example, D has index 0 in the key, G index 1 etc. If the original message contains a B (character code 66), it is decreased by 65 to the value 1. This value is used as index in the key, which gives the character G.

Here is the code:

```
#include <iostream.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
void main()
{
```


Download free eBooks at bookboon.com

```
  int i, j, iLen;
  char cTemp, cText[50], cEncrypt[50];
  char cKey[27] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
  srand(time(0));
  for (i=0; i<26; i++)
  {
    j = rand() % 26;
    cTemp = cKey[i];
    cKey[i] = cKey[j];
    cKey[j] = cTemp;
  }
  cout << "Write a text in upper case: " << endl;
  cin.getline(cText,49);
  iLen = strlen(cText);
  for (i=0; i<iLen; i++)
    cEncrypt[i] = cKey[cText[i] - 65];
  cEncrypt[iLen] = '\0';
  cout << "Encrypted: " << cEncrypt << endl;
}
```

First we declare some supporting variables; i, j and iLen, the latter being used for the length of the entered message. The string array cText will store the entered text, and cEncrypt for the encrypted message. The string array cKey is initated with the characters A-Z. Then the random number generator is initiated.

The first for-loop performs the mixing of the characters of the array cKey. The loop counter i goes from 0 to 25 and points to character by character in cKey. A j value is randomly created to be used as index to the character to be interchanged with the i-character. Then the triangular exchange of these two characters is performed. At completion of the loop all characters in cKey has been interchanged with some other character, which makes cKey now be well-mixed.

The user is prompted for a message that is stored in the string array cText. The length of the message is calculated and stored in the variable iLen.

The second for-loop performs the translation to an encrypted message. The loop counter i goes from 0 to iLen-1, i.e. as many turns as there are characters in the input string. In the statement

```
cEncrypt[i] = cKey[cText[i] - 65];
```

a character is taken from cText (cText[i]).. This value is decreased by 65, and since it is a math operation, the result is of integer type. If the character is B (character code 66), the calculation 66-65 is performed which gives 1. This value is now used as index in cKey and gives the character from cKey that corresponds to the original position of B, for instance G. This character is stored in the string array cEncrypt at position i. The array cEncrypt is thus built up by one character at a time from the array cText, translated via cKey. At completion of the loop all characters in cText have been translated via cKey and been stored in cEncrypt.

Download free eBooks at bookboon.com

After the last character in cEncrypt we add the null character. The array cEncrypt containing the encrypted message is then printed.

5.22 Summary

In this chapter we have introduced the char data type. A char variable can contain only one character. We have also learnt how to use string arrays or string variables to store texts consisting of more than one character.

We have also shown how to use character codes to manipulate texts, for instance to convert between upper and lower case.

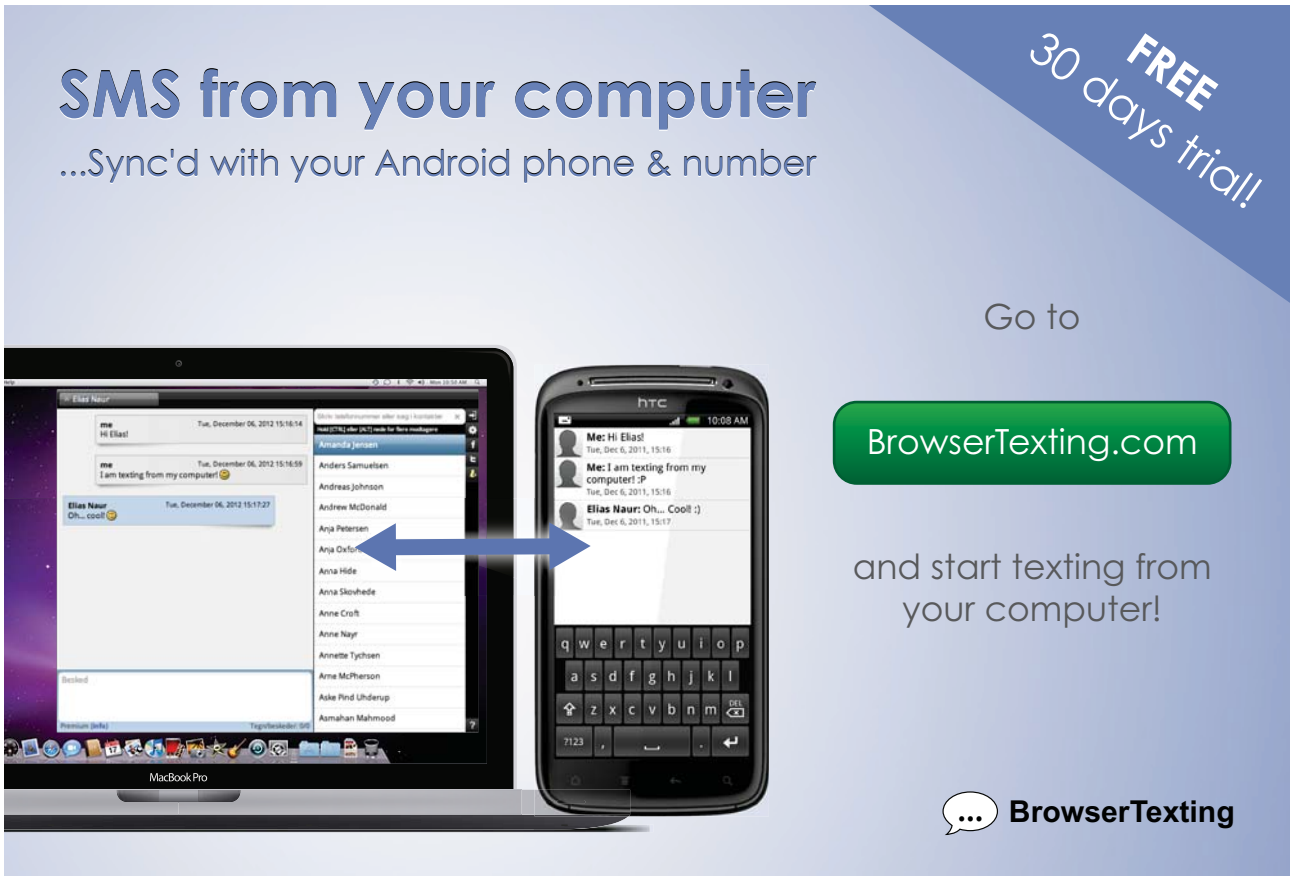
Blanks is a problem at entry of texts, which is solved by using the function getline().

The header file string.h contains a number of nice functions for text management, for instance to calculated the length of a text, copy texts, concatenate texts and extract substrings of texts. We have also created a primary relation to pointers, which we will return to more in detail in a later chapter.

We have also worked through a number of programming examples to extract initials from a name, sort a list of names, where we got use of two-dimensional arrays (matrices), interchange first name and surname, encrypt messages and create random passwords.

5.23 Exercises

1. Start from the menu program earlier in this chapter and add an extra option to the menu:
   `E. Statistics`
   Complete the switch statement with a suitable text.



Click on the ad to read more
Download free eBooks at bookboon.com

2. Write a menu program where you by means of entry of a character can select to get either your name, address, postal address or telephone number printed. It should be performed with a loop so that you repeatedly can select from the menu until you enter X to exit the program.
3. Write a program that asks for a character and then prints the character 10 times at the same line.
4. Change the previous program so that the program prints the character once at the first line, twice at the second line etc. to 10 times at the tenth line.
5. Write a program that prompts the user for a character and an integer. The program should then print the character as many times as given by the entered integer.
6. Change the Christmas Tree program in this chapter so that it prints 15 branches instead of 11.
7. Write a program that prompts the user for a character and prints the corresponding character code.
8. Complete the previous program with entry of a character code and a printout of the corresponding character.
9. Write a program that prints the text 'Här går vi över ån efter vatten' with the Swedish characters å, ä and ö in a correct way.
10. Write a program that reads a text from the keyboard and prints the length of the text as well as the entire declared size of the string variable.
11. Start with the Upper/Lower Case program and change it so that it reads a text in upper case and prints it in lower case. What happens if you enter other characters. Explain why the printout is the way it is.
12. Write a program that prompts the user for a word and prints each character doubled (twice).
13. Write a program that prompts the user for a word and prints it reversed, first by using the strrev() function, and then without strrev().
14. Start from the Initials program in this chapter and complete it so that the user can enter first name, middle name and surname and get all three initials printed.
15. Complete the previous program so that, if you enter the names with leading lower case characters, the printout will still be with the initials in upper case.
16. Write a program that prompts the user for two characters and prints all characters in between. For instance, if you enter F and K, the printout should be FGHIJK.
17. Write a program that prompts the user for two words and prints the one first in alphabetic order.
18. Write a program that prompts the user for a word in lower case. The program should respond with the number of vowels in the word, Don't include å, ä or ö.
19. The Robbery language is spoken by doubling each consonant and putting an 'o' in between. For instance the word 'bug' becomes 'bobugog'. Write a program that reads a word and prints it in the Robbery language.
20. Write a program where the user can enter 6 product names. Sort the list and print it.
21. Complete the previous program so that the user also can enter the prices of the 6 products. The printout should contain the correct price for each product, right-aligned with two decimals.
22. Write a program where the user can enter his first name and surname. The program should then print only the surname.
23. Write a program where the user can enter his e-mail address. The program should then check whether there is an @ in the address.
24. Complete the previous program so that, if there is no @, the text '@htu.se' is added and the address is printed.
25. Suppose that a person enters his e-mail address with a period between the first name and the surname, e.g.: john.smith@htu.se

Download free eBooks at bookboon.com

Write a program that reads an e-mail address and prints the name in the correct way, e.g.:
John Smith
with upper case first character in both first and surname and with a blank in between.

26. Start with the Encryption program in this chapter. Change so that B becomes A, C becomes B etc.
27. Change the previous program so that a character is replaced by the one 3 positions earlier in the alphabet.
28. Write a program that decrypts instead of encrypts according to the method in the previous exercise.
29. Write a program that encrypts a text by reversing the alphabet, i.e. A becomes Z, B becomes Y etc.
30. Start with the Random Password program in this chapter. Complete it with also taking lower case characters and digits into account.
31. Change the previous program so that the password will be 6-10 characters in size.
32. Originate from the Translation Table program in this chapter. Complete it with also taking lower case characters into account.
33. Use your imagination to construct an own encryption algorithm and write a program for it.

Click on the ad to read more

Download free eBooks at bookboon.com